Engenharia de Software

16/3/2026

# TP3

## Velocidade e Qualidade com Estruturas de Dados e Algoritmos

Professor(a): Ricardo Pires Mesquita

Samuel Hermany
INSTITUTO INFNET

## LINK PARA O VÍDEO

https://drive.google.com/file/d/1G_wz5Lr6qBfFDZ_QAREpyQz_dge3LQ9h/view

## 1. QUESTÃO

A soma ocorre no momento do retorno das chamadas recursivas,
pois a função primeiro chama mult(n1, n2-1) até chegar ao caso base.
Somente quando as chamadas começam a retornar é que a soma n1 + resultado é realizada.

### 1 - Multiplicação Recursiva por Soma

```python
def produto(n1, n2):
    print(f"Chamada produto({n1} x {n2})")

    # verifica se algum número é negativo
    if n1 < 0 or n2 < 0:
        print(f"Não é permitido números negativos")
        return None

    # CASO BASE: quando n2 chega a 0 a recursão termina
    if n2 == 0:
        print("Return 0")
        return 0

    # soma n1 com o resultado da próxima chamada recursiva
    resultado = n1 + produto(n1, n2 - 1)
    print(f"Retorno {resultado} para produto({n1} x {n2})")
    return resultado   # resultado da multiplicação


# TESTANDO
n1 = 2      # primeiro número da multiplicação
n2 = 3      # segundo número da multiplicação
resultado = produto(n1, n2)
print(f'\nA multiplição por soma de {n1} x {n2} = {resultado}')
# observação: a soma ocorre no retorno
```

```
Chamada produto(2 x 3)
Chamada produto(2 x 2)
Chamada produto(2 x 1)
Chamada produto(2 x 0)
Return 0
Retorno 2 para produto(2 x 1)
Retorno 4 para produto(2 x 2)
Retorno 6 para produto(2 x 3)

A multiplição por soma de 2 x 3 = 6
```

## 2. QUESTÃO

### 2 - Fatoração Recursiva de Inteiros

[47]
0s

```python
import math

# x = número que queremos fatorar
# lowest = menor divisor que vamos testar
def factor(x, lowest=2):

    # CASOS ESPECIAIS
    if x == 0 or x ==1 or x < 0:
        return "Casos especiais"

    # limite de busca até a raiz quadrada
    limite = int(math.sqrt(x))

    # testa divisores entre lowest e √x
    for i in range(lowest, limite + 1):
        # se encontrou divisor com resto = 0
        if x % i == 0:
            # adiciona o fator e continua fatorando o resto
            # "x // i" dividie o número pelo fator encontrado → será o novo x
            # ", i)" passa a ser o novo lowest (menor divisor a testar)
            # "+" junta as duas istas de fatores
            return [i] + factor(x // i, i)

    # se nenhum divisor foi encontrado → número primo
    return [x]


# TESTANDO
testes = [72, 64, 102, 125, 18, 36, 24]

for n in testes:
    print(f"Fatores de {n} -> {factor(n)}")
```

```
Fatores de 72 -> [2, 2, 2, 3, 3]
Fatores de 64 -> [2, 2, 2, 2, 2, 2]
Fatores de 102 -> [2, 3, 17]
Fatores de 125 -> [5, 5, 5]
Fatores de 18 -> [2, 3, 3]
Fatores de 36 -> [2, 2, 3, 3]
Fatores de 24 -> [2, 2, 2, 3]
```

## 3. QUESTÃO

**Análise de complexidade:**
A cada chamada recursiva o expoente é reduzido em 1 até chegar ao caso base. Portanto, o número de chamadas é proporcional ao valor absoluto do expoente. Assim, a complexidade de tempo é O(|y|) e a profundidade da pilha de recursão também é O(|y|).

### 3 - Potenciação Recursiva com Expoentes Negativos

[26]
0s

```python
def power(x, y):
    # CASOS BASE
    if x == 0 and y < 0:
        return "Indefinido"

    if y == 0:
        return 1

    if y == 1:
        return x

    # expoente negativo
    if y < 0:
        return 1 / power(x, -y)  # transforma o expoente -y para y
    else:
        return x * power(x, y - 1 )  # "power()" chama a funçã ocom um expoente menor

# TESTANDO
lista_bases = [0, 1, 2, 5, 8]      # bases
lsita_potencia = [-2, 0, 1, 2, -3]    # expoentes

for i in lista_bases:
    for potencia in lsita_potencia:
        print(f"Potência de {i}^{potencia} = {power(i, potencia)}")

# Complexida O(y)
```

```
Potência de 0^-2 = Indefinido
Potência de 0^0 = 1
Potência de 0^1 = 0
Potência de 0^2 = 0
Potência de 0^-3 = Indefinido
Potência de 1^-2 = 1.0
Potência de 1^0 = 1
Potência de 1^1 = 1
Potência de 1^2 = 1
Potência de 1^-3 = 1.0
Potência de 2^-2 = 0.25
Potência de 2^0 = 1
Potência de 2^1 = 2
```

# 4. QUESTÃO

## 4 - Navegação Recursiva em Estrutura de Diretórios

[2]

```python
# 1 - listar itens da pasta
# 2 - para cada item:
#      se for arquivo → mostrar
#      se for pasta → mostrar e entrar nela (recursão)
import os

def navegar(diretorio, nivel=0):
    # lista todos os itens(diretótios/pastas) dentro do diretório
    itens = os.listdir(diretorio)

    # percorre todos os itens(diretótios/pastas) dentro
    for item in itens:
        # nome da pasta(diretorio = "docs") + nome do arquivo(item =
        "arquivo.txt") ou subpasta = caminho completo
        caminho = os.path.join(diretorio, item)
        print("  " * nivel + item)      # indentação conforme o nível

        # se for pasta, entra nela
        if os.path.isdir(caminho):
            navegar(caminho, nivel + 1)  # (recursão)

# Testando
navegar("/content")
```

```
.config
  hidden_gcloud_config_universe_descriptor_data_cache_configs.db
  .last_opt_in_prompt.yaml
  .last_survey_prompt.yaml
  configurations
    config_default
  logs
    2026.01.16
      14.24.03.314209.log
      14.24.29.392089.log
      14.24.28.646070.log
      14.23.31.981136.log
      14.24.18.954466.log
      14.24.13.071214.log
  config_sentinel
  active_config
  gce
  default_configs.db
```

```
    default_configs.db
    .last_update_check.json
drive
  Shareddrives
  .shortcut-targets-by-id
    1mT01eWlI0x2Nt80kXl4oHImghaItPnur
      Publico
        DBE_PB_OrientacoesProjeto.gdoc
        DFE_PB_ProjetoWheels_Ter.gsheet
        DBE_PB_Apresentacoes.gdoc
        Cópia de DBE_PB_Apresentacoes.gdoc
        DFE_PB_DocumentoDoProjeto_Wheels.gdoc
  MyDrive
    Colab Notebooks
      .ipynb_checkpoints
      Excel e Ações Testes
        dados_META.xlsx
        dados_MSFT.xlsx
        dados_GOOGL.xlsx
        dados_ORLY.xlsx
        Backtestes_Ações.ipynb
        Backtestes_Opções.ipynb
        Backtestes_Ações_original.ipynb
      Copy of Untitled0.ipynb
      teste grtaficos.ipynb
    Cursos Livres
      Desenvolvimento Ágil - DevOps Docker.docx
      Consultando Bases SQL com Python 1.mp4
      Consultando Bases SQL com Python 2.mp4
      Introdução aos dispositivos wearables.mp4
      Internet das Coisas IOT.mp4
      Topicos de UX para Design de Servicos.mp4
      04 09 23 - Programando para Web com Python, CSS e HTML.mp4
      06 09 23 - Programando para Web com Python, CSS e HTML.mp4
      11 09 23 - Programando para Web com Python, CSS e HTML.mp4
      30 08 23 - Programando para Web com Python, CSS e HTML.mp4
      13 09 23 - Prototipação no-code de soluções digitais (Parte 1).mp4
      Programando para Web com Python, CSS e HTML (parte 4).pdf
      Prototipação no-code de soluções digitais (Parte 1).pdf
    Documentos
      INFNET-Desenvolvimento Mobile com React Native.docx
      INFNET-Desenvolvimento Web com React.docx
      INFNET-Fundamentos de Desenvolvimento com Java.docx
      INFNET-Fundamentos de Desenvolvimento com C#.docx
      INFNET-PB Desenvolvimento Back End.docx
      INFNET-Desenvolvimento Web com .NET e Bases de Dados.docx
      INFNET-Desenvolvimento de Serviços Web e Testes com Java.docx
      INFNET-Desenvolvimento de Serviços com SpringBoot.docx
      INFNET-Design Patterns e Domain-Driven Design(DDD) com Java.docx
```

# 5. QUESTÃO

## 5 - Geração Recursiva de Combinações (Times)

[22]
✓ 0s

```python
def teams(candidates, k, inicio=0, atual=""):
    # verifica se o time atual já tem k(quantidade de pessoas p/ completar o time)
    # len(atual.split()): separa o time atual ple ospaço entre as palavras e entao c
    if len(atual.split()) == k:
        print(atual.strip())    # mostra o time formado
        return

    # percorre os candidatos disponíveis na lista
    for i in range(inicio, len(candidates)):
        # recursividasde, adicionao o candidato atual ao time
        # i + 1 garante que não vamos repetir candidatos anteriores
        # candidates, k, i + 1 = inicio, atual = lista atual já formada
        teams(candidates, k, i + 1, atual + " " + candidates[i])


# TESTANDO-------------------------------------------------------
candidates = ["Ana", "Bruno", "Carlos", "Diana", "Ricado"]
k = 2
teams(candidates, k)
```

```
Ana Bruno
Ana Carlos
Ana Diana
Ana Ricado
Bruno Carlos
Bruno Diana
Bruno Ricado
Carlos Diana
Carlos Ricado
Diana Ricado
```

```python
# TESTANDO--------------------
candidates = ["Ana", "Bruno",
k = 3
teams(candidates, k)
```

```
Ana Bruno Carlos
Ana Bruno Diana
Ana Bruno Ricado
Ana Carlos Diana
Ana Carlos Ricado
Ana Diana Ricado
Bruno Carlos Diana
Bruno Carlos Ricado
Bruno Diana Ricado
Carlos Diana Ricado
```

```python
# TESTANDO--------------------
candidates = ["Ana", "Bruno",
k = 4
teams(candidates, k)
```

```
Ana Bruno Carlos Diana
Ana Bruno Carlos Ricado
Ana Bruno Diana Ricado
Ana Carlos Diana Ricado
Bruno Carlos Diana Ricado
```

## 6. QUESTÃO

### 6 - Problema da Mochila (Knapsack Recursivo)

```python
def knapsack(target, weights, index=0, atual=None, resultados=None):
    if atual is None:       # cria lista vazia para armazenar pesos escolhidos
        atual = []

    if resultados is None:  # cria lista para armazenar todas as combinações válidas
        resultados = []

    # CASO BASE: se o target chegar a 0, encontramos uma combinação válida
    if target == 0:
        resultados.append(atual.copy())  # guarda a combinação encontrada
        return resultados

    # CASO BASE: se o target ficar negativo ou acabar os pesos
    if target < 0 or index >= len(weights):
        return resultados  # encerra essa tentativa

    # 1: adiciona peso atual na combinação
    # target: weights[index]: diminui o target pelo peso escolhido ,
    # weights: lsita de pessos
    # index + 1: cham o proximo índice pro peso
    #  atual + [weights[index]]: adiciona o peso atual na lista
    knapsack(target - weights[index], weights, index + 1, atual + [weights[index]],
    resultados)

    # 2: NÃO adiciona peso atual na combinação
    knapsack(target, weights, index + 1, atual, resultados)

    return resultados   # retorna todas as combinações encontradas


# TESTANDO ----------------------------------------
weights = [1, 2, 3, 4, 5, 7]    # pesos disponíveis
target = 10                     # valor que queremos atingir

resultado = knapsack(target, weights)
print("Combinações que somam", target)  # combinações encontradas
```

```python

# precorre todos os resultados
if len(resultado) == 0:
    print("Nenhuma combinação encontrada")
else:
    for i in resultado:
        print(i)
```

```
Combinações que somam 10
[1, 2, 3, 4]
[1, 2, 7]
[1, 4, 5]
[2, 3, 5]
[3, 7]
```

## 7. QUESTÃO

A questão escolhida foi a questão 1.
**Análise de Complexidade da Função produto**

- Chamada redundante: Não existem chamadas redundantes; cada chamada recursiva é necessária.
- Crescimento do número de chamadas: A função é chamada recursivamente uma vez para cada decremento de n2 até atingir 0. Se n2 = 8, haverá 8 chamadas e assim para qualquer outro valor de n, portanto o número de chamadas cresce linearmente com n2.
- Complexidade (Big O): **O(n²)**, pois o número de chamadas e operações é proporcional a n2.

7 - Análise de Complexidade em Algoritmos Recursivos

[25] 0s

```python
def produto(n1, n2):
    # verifica se algum número é negativo
    if n1 < 0 or n2 < 0:
        return None

    # CASO BASE: quando n2 chega a 0 a recursão termina
    if n2 == 0:
        return 0

    # soma n1 com o resultado da próxima chamada recursiva
    resultado = n1 + produto(n1, n2 - 1)
    return resultado   # resultado da multiplicação


# TESTANDO
n1 = 3      # primeiro número da multiplicação
n2 = 3      # segundo número da multiplicação
resultado = produto(n1, n2)
print(f'\nA multiplição por soma de {n1} x {n2} = {resultado}\n')

print("""A questão escolhida foi a questão 1 (Multiplicação Recursiva por Soma).

ANÁLISES:
Chamada redundante: Não existem chamadas redundantes; cada chamada recursiva é necessária para somar n1, n2 vezes.
Crescimento do número de chamadas: A função é chamada recursivamente uma vez para cada decremento de n2 até atingir 0. Se n2 = 5, haverá 5 chamadas. O número de chamadas cresce linearmente com n2.
Complexidade (Big O): O(n2), pois o número de chamadas e operações é proporcional a n2, ou seja, a complexidade cresce segundo o valor de n2.""")
```

```
A multiplição por soma de 3 x 3 = 9

A questão escolhida foi a questão 1 (Multiplicação Recursiva por Soma).

ANÁLISES:
Chamada redundante: Não existem chamadas redundantes; cada chamada recursiva é necessária para somar n1, n2 ve
Crescimento do número de chamadas: A função é chamada recursivamente uma vez para cada decremento de n2 até at
Complexidade (Big O): O(n2), pois o número de chamadas e operações é proporcional a n2, ou seja, a complexidad
```

# 8. QUESTÃO

## 8.3. Comparação do número de chamadas

Observando os resultados do testes de **Fib(3)** até **Fib(10)** ,podemos observar que a versão recursiva simples realiza muito mais chamadas de função, pois recalcula várias vezes os mesmos valores de Fibonacci.
Já a versão com programação dinâmica armazena os resultados já calculados (memorização), evitando refazer cálculos desnecessários.

## 8.4. Análise de redução da complexidade

Na versão recursiva a complexidade aproximada é O(2^n), pois cada chamada gera duas novas chamadas recursivas.
Já na versão com programação dinâmica a complexidade é O(n), pois cada valor de Fibonacci é calculado apenas uma vez e depois reutilizado a partir do dicionário de memoização.

## 8.5 Evidências de melhoria

A melhoria pode ser observada diretamente na tabela abaixo onde a razão da diminuição do número de chamadas reduz tanto quanto o valor de "**n**" cresce.
Ou seja, a versão com programação dinâmica reduz drasticamente o número de chamadas e o custo computacional, mantendo exatamente o mesmo resultado da sequência de Fibonacci.

| Fib(n) = resultado | Chamadas Recursivo | Chamadas Prog. Dinâmica |
|---|---|---|
| Fib(3) = 2 | 3 | 3 |
| Fib(5) = 5 | 9 | 7 |
| Fib(6) = 8 | 15 | 9 |
| Fib(8) = 21 | 41 | 13 |
| Fib(10) = 55 | 109 | 17 |

## 8 - Otimização com Programação Dinâmica

### 8.1. - Versão recursiva

[5]
✓ 0s

```python
# Recusividade Direta - "n" representa a posição na sequência de Fibonacci que queremos calcular
def fibonacci_rec(n):
    # CASOS BASE da recursividade, na sequência de Fibonacci: F(1) e F(2) = 1
    if n == 1 or n == 2:
        return 1            # retorna 1 porque os dois primeiros valores da sequência são 1
    else:
        # valor atual é a soma dos dois anteriores
        # função chama ela mesma duas vezes, p/ calcular F(n-1) e p/ calcular F(n-2)
        return fibonacci_rec(n-1) + fibonacci_rec(n-2)
```

## 8.2. - Programação Dinâmica

[58]
✓ 0s

```python
# Fibonacci com Programação Dinâmica (Memoização)
def fibonacci_pd(n, memo=None, chamadas=None):
    if memo is None:
        memo = {}  # dicionário para armazenar resultados já calculados
    if chamadas is None:
        chamadas = {"total": 0}  # contador de chamadas

    chamadas["total"] += 1  # registra cada chamada

    # se o valor já foi calculado, retorna do memo
    if n in memo:
        return memo[n], chamadas["total"]

    # casos base
    if n == 1 or n == 2:
        memo[n] = 1
        return 1, chamadas["total"]

    # chama recursivamente com memo
    a, _ = fibonacci_pd(n-1, memo, chamadas)
    b, _ = fibonacci_pd(n-2, memo, chamadas)

    memo[n] = a + b
    return memo[n], chamadas["total"]
```

## 8.3 - Test

[57]
✓ 0s

```python
# TESTE COMPARATIVO ----------------------------------------------------
lista = [3, 5, 6, 8, 10]
print("COMPARAÇÃO FIBONACCI:")
print(f"{'':<16}Recursivo{'':<4}vs {'':<4}Programação Dinâmica")

for item in lista:
    # Recursão Simples
    for i in range(1, item + 1):
        resultado_rec, total_chamadas_rec = fibonacci_rec(i)

    # Com Memoização
    for i in range(1, item + 1):
        resultado_pd, total_chamadas_pd = fibonacci_pd(i)

    print(f"{f'Fib({item}) = {resultado_rec}':<15} chamadas {total_chamadas_rec:<3} |
    {total_chamadas_pd} chamadas")
```

```
COMPARAÇÃO FIBONACCI:
                Recursivo    vs     Programação Dinâmica
Fib(3) = 2      chamadas 3   | 3 chamadas
Fib(5) = 5      chamadas 9   | 7 chamadas
Fib(6) = 8      chamadas 15  | 9 chamadas
Fib(8) = 21     chamadas 41  | 13 chamadas
Fib(10) = 55    chamadas 109 | 17 chamadas
```

## 9. QUESTÃO

Com o pivô como último elemento, os testes mostram que listas aleatórias foram as mais rápidas, enquanto listas ordenadas e invertidas foram mais lentas. Isso ocorre porque o algoritmo usa o último elemento como pivô. Em listas aleatórias, o pivô tende a dividir o array de forma mais equilibrada, aproximando o desempenho de O(n log n). Já em listas ordenadas ou invertidas, o pivô geralmente vira o maior ou menor elemento, gerando partições desbalanceadas. Nesses casos, o QuickSort se aproxima do pior caso O(n²), o que explica os tempos maiores observados.

**ANÁLISE DE COMPORTAMENTO:**
Os testes mostram que listas aleatórias foram as mais rápidas, enquanto listas ordenadas e invertidas foram mais lentas. Isso ocorre porque o algoritmo usa o último elemento como pivô. Em listas aleatórias, o pivô tende a dividir o array de forma mais equilibrada, aproximando o desempenho de O(n log n). Já em listas ordenadas ou invertidas, o pivô geralmente vira o maior ou menor elemento, gerando partições desbalanceadas. Nesses casos, o QuickSort se aproxima do pior caso O(n²), o que explica os tempos maiores observados.

### 9 - Implementação do QuickSort

```python
import time
import random
def quick_sort(lista):
    # CASO BASE: se a lista tem 0 ou 1 elemento, ela já está ordenada
    if len(lista) <= 1:
        return lista

    pivo = lista[len(lista)-1]     # Escolhe o último elemento como pivô

    # Cria listas menores com list comprehension
    menores = [x for x in lista[:-1] if x <= pivo]

    # Cria listas maiores com list comprehension
    maiores = [x for x in lista[:-1] if x > pivo]

    # dividir + resolver recursivamente + combinar
    return quick_sort(menores) + [pivo] + quick_sort(maiores)


# TESTANDO
listas = [
    ("lista ordenada", list(range(1, 21, 1))),
    ("lista ordenada", list(range(20, 41, 1))),
    ("lista invertida", list(range(60, 0, -2))),
    ("lista invertida", list(range(80, 0, -3))),
    ("lista aleatória", random.sample(range(1, 100), 20)),
    ("lista aleatória", random.sample(range(1, 100), 20))
]

resultados = []

for i, (nome, lista) in enumerate(listas, start=1):
    print(f"{i} - {nome}:")

    inicio_t = time.perf_counter()
    lista_ordenada = quick_sort(lista)
    tempo = time.perf_counter() - inicio_t

    print(f"\tTempo: {tempo:.8f} segundos = {lista_ordenada}")
    resultados.append((nome, tempo))
```

[17]
✓ 0s

```python
    resultados.append((nome, tempo))
# ========================
# ANÁLISE FINAL
# ========================

print("\nANÁLISE DE DESEMPENHO\n")

# Ordena do mais rápido para o mais lento
resultados.sort(key=lambda x: x[1])

melhor_tempo = resultados[0][1]

for i, (nome, tempo) in enumerate(resultados, start=1):
    diferenca = ((tempo - melhor_tempo) / melhor_tempo) * 100

    if i == 1:
        print(f"{i}º MAIS RÁPIDO → {nome} | tempo = {tempo:.8f} s | referência (0%)")
    else:
        print(f"{i}º → {nome} | tempo = {tempo:.8f} s | {diferenca:.2f}% mais lento")
```

```
1 - lista ordenada:
        Tempo: 0.00009122 segundos = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16
2 - lista ordenada:
        Tempo: 0.00004536 segundos = [20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 3
3 - lista invertida:
        Tempo: 0.00023884 segundos = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 3
4 - lista invertida:
        Tempo: 0.00005880 segundos = [2, 5, 8, 11, 14, 17, 20, 23, 26, 29, 32, 35, 38, 41,
5 - lista aleatória:
        Tempo: 0.00002299 segundos = [4, 8, 11, 13, 18, 23, 26, 35, 45, 52, 55, 57, 63, 65,
6 - lista aleatória:
        Tempo: 0.00002392 segundos = [1, 11, 16, 32, 37, 42, 50, 53, 54, 57, 62, 63, 66, 73

ANÁLISE DE DESEMPENHO

1º MAIS RÁPIDO → lista aleatória | tempo = 0.00002299 s | referência (0%)
2º → lista aleatória | tempo = 0.00002392 s | 4.05% mais lento
3º → lista ordenada | tempo = 0.00004536 s | 97.35% mais lento
4º → lista invertida | tempo = 0.00005880 s | 155.82% mais lento
5º → lista ordenada | tempo = 0.00009122 s | 296.88% mais lento
6º → lista invertida | tempo = 0.00023884 s | 939.11% mais lento
```

## 10. QUESTÃO

Na comparação de cenários com diferentes entradas podemos ver que listas ordenadas e invertidas realizam mais comparações e cópias. Isso ocorre porque o pivô é sempre o último elemento, gerando divisões muito desbalanceadas. Assim, o algoritmo precisa percorrer quase toda a lista em cada chamada recursiva. Já nas listas aleatórias, as divisões tendem a ser mais equilibradas. Por isso, o número de operações e o tempo de execução são menores.

Isso aproxima o comportamento do pior caso do QuickSort $O(n^2)$, enquanto listas aleatórias tendem a ficar mais próximas do caso médio $O(n \log n)$.

### 10 - Instrumentação do QuickSort

[7]
0s

```python
import time
import random

comparacoes = 0
copias = 0
def quick_sort(lista):
    global comparacoes, copias      # impede que as variáveis sejam recriadas localmente a durante a recursão
    menores = []
    maiores = []

    # CASO BASE: se a lista tem 0 ou 1 elemento, ela já está ordenada
    if len(lista) <= 1:
        return lista

    pivo = lista[len(lista)-1]     # escolhe o último elemento como pivô

    for x in lista[:-1]:
        comparacoes += 1
        # se testamos algo é uma comparação
        if x <= pivo:
            menores.append(x)
            copias += 1  # se adicionamos um elemento na lista estamos fazendo cópias
        else:
            maiores.append(x)
            copias += 1  # se adicionamos um elemento na lista estamos fazendo cópias

    # juntar listas, também é cópia
    copias += len(menores) + len(maiores) + 1

    # dividir + resolver recursivamente + combinar
    return quick_sort(menores) + [pivo] + quick_sort(maiores)


# TESTANDO
listas = [
    ("lista ordenada", list(range(1, 21, 1))),
    ("lista ordenada", list(range(20, 41, 1))),
    ("lista invertida", list(range(60, 0, -2))),
    ("lista invertida", list(range(80, 0, -3))),
    ("lista aleatória", random.sample(range(1, 100), 20)),
    ("lista aleatória", random.sample(range(1, 100), 20))
]

resultados = []
```

```python
resultados = []

for i, (nome, lista) in enumerate(listas, start=1):
    comparacoes = 0
    copias = 0

    print(f"{i} - {nome}:")

    inicio = time.perf_counter()
    quick_sort(lista)
    tempo = time.perf_counter() - inicio

    print(f"\tTempo: {tempo:.8f} s")
    print(f"\tComparações: {comparacoes}")
    print(f"\tCópias: {copias}")

    resultados.append((nome, tempo, comparacoes, copias))
```

```
1 - lista ordenada:
        Tempo: 0.00006202 s
        Comparações: 190
        Cópias: 399
2 - lista ordenada:
        Tempo: 0.00004248 s
        Comparações: 210
        Cópias: 440
3 - lista invertida:
        Tempo: 0.00021609 s
        Comparações: 435
        Cópias: 899
4 - lista invertida:
        Tempo: 0.00006466 s
        Comparações: 351
        Cópias: 728
5 - lista aleatória:
        Tempo: 0.00002195 s
        Comparações: 81
        Cópias: 175
6 - lista aleatória:
        Tempo: 0.00001769 s
        Comparações: 64
        Cópias: 139
```

# 11. QUESTÃO

## 11 - Implementação do QuickSelect

[4]
0s

```python
def particao(lista, esq, dir):
    pivo = lista[dir]  # escolhe o último elemento da lista como pivô
    i = esq - 1        # índice que indica a posição final dos elementos menores que o pivô

    # percorre a lista do índice 'esq' até 'dir'
    for j in range(esq, dir):
        # se o elemento atual for menor ou igual ao pivô, menores a esquerda
        if lista[j] <= pivo:
            i += 1  # avança o índice dos menores
            # troca o elemento atual com o elemento na posição i
            # isso move o valor menor que o pivô para a parte esquerda da lista
            lista[i], lista[j] = lista[j], lista[i]

    # após percorrer todos os elementos, coloca o pivô na posição correta
    # todos os elementos menores ficam à esquerda e os maiores à direita
    lista[i + 1], lista[dir] = lista[dir], lista[i + 1]

    # retorna o índice final onde o pivô foi colocado
    return i + 1


# Função principal do QuickSelect que encontra o k-ésimo menor elemento
def quickselect(lista, esq, dir, k):
    if esq <= dir:      # verifica se ainda existe uma parte válida da lista para analisar

        # chama a função de partição para reorganizar a lista e obter a posição final do pivô
        pivo_index = particao(lista, esq, dir)

        # sinterrompe ao encontrae o k-ésimo menor elemento
        if pivo_index == k:
            return lista[pivo_index]

        # se a posição do pivô for maior que k, então o elemento procurado está na parte esquerda da lista
        elif pivo_index > k:
            return quickselect(lista, esq, pivo_index - 1, k)
        else:                        # caso contrário, o elemento está na parte direita da lista
            return quickselect(lista, pivo_index + 1, dir, k)

    return None         # caso a busca não encontre o elemento (situação limite)


# TESTANDO ----------------------------------------------------------------------
teste = [6, 4, 7, 1, 3, 8, 5, 2]
mediana = len(teste) // 2
resultado = quickselect(teste, 0, len(teste) - 1, mediana)

print("Mediana:", resultado)
```

```
Mediana: 5
```

# 12. QUESTÃO

## 12.3. Análise dos resultados obtidos

- **Quick Sort** sempre ordena toda a lista, então o número de comparações e cópias é maior, já o **Quick Select** realiza menos comparações e cópias, porque interrompe a recursão assim que encontra o elemento desejado.
- Em listas ordenadas, Quick Select reduziu significativamente o número de cópia e em listas invertidas, **Quick Select** ainda fez todas as comparações necessárias em alguns casos, mas realizou muito menos cópias
- Em listas aleatórias, **Quick Select** apresentou maior eficiência, com menor tempo de execução, comparações e cópias

## 12.4. Cenários mais adequados para cada algoritmo

- QuickSort: ideal quando é necessário ordenar toda a lista, ou quando se precisa de todos os elementos em ordem.
- QuickSelect: funciona melhor quando queremos um elemento específico, como a mediana ou o k-ésimo menor valor.

## 12.5. Conclusões técnicas

- QuickSelect é mais eficiente em termos de tempo e número de cópias se o objetivo é um único elemento.
- QuickSort é mais custoso, mas fornece informação completa da lista.
- Em listas já ordenadas ou invertidas, **Quick Select** se beneficia da lógica de interrupção, reduzindo operações desnecessárias.
- Para listas aleatórias pequenas, a diferença de tempo é pequena, mas **Quick Select** ainda apresenta menos operações e menor custo.
- Escolher o algoritmo certo depende do objetivo do problema: selecionar um valor ou ordenar toda a lista.

12 - Comparação entre QuickSort e QuickSelect

12.1. - QuickSort

[7]
0s

```python
import time
import random

comparacoes = 0
copias = 0
def quick_sort(lista):
    global comparacoes, copias      # impede que as variáveis sejam recriadas localmente a durante a recursão
    menores = []
    maiores = []

    # CASO BASE: se a lista tem 0 ou 1 elemento, ela já está ordenada
    if len(lista) <= 1:
        return lista

    pivo = lista[len(lista)-1]     # escolhe o último elemento como pivô

    for x in lista[:-1]:
        comparacoes += 1
        # se testamos algo é uma comparação
        if x <= pivo:
            menores.append(x)
            copias += 1  # se adicionamos um elemento na lista estamos fazendo cópias
        else:
            maiores.append(x)
            copias += 1  # se adicionamos um elemento na lista estamos fazendo cópias

    # juntar listas, também é cópia
    copias += len(menores) + len(maiores) + 1

    # dividir + resolver recursivamente + combinar
    return quick_sort(menores) + [pivo] + quick_sort(maiores)
```

## 12.2. Quick Select

[9]
✓ 0s

```python
comparacoes = 0
copias = 0

def particao(lista, esq, dir):
    global comparacoes, copias

    pivo = lista[dir]  # escolhe o último elemento da lista como pivô
    i = esq - 1        # índice que indica a posição final dos elementos menores que o pivô

    # percorre a lista do índice 'esq' até 'dir'
    for j in range(esq, dir):
        comparacoes += 1
        # se o elemento atual for menor ou igual ao pivô
        if lista[j] <= pivo:
            i += 1  # avança o índice dos menores
            # troca o elemento atual com o elemento na posição i
            # isso move o valor menor que o pivô para a parte esquerda da lista
            lista[i], lista[j] = lista[j], lista[i]
            copias += 1

    # após percorrer todos os elementos, coloca o pivô na posição correta
    # todos os elementos menores ficam à esquerda e os maiores à direita
    lista[i + 1], lista[dir] = lista[dir], lista[i + 1]
    copias += 1

    # retorna o índice final onde o pivô foi colocado
    return i + 1


# Função principal do QuickSelect que encontra o k-ésimo menor elemento
def quickselect(lista, esq, dir, k):
    if esq <= dir:      # verifica se ainda existe uma parte válida da lista para analisar

        # chama a função de partição para reorganizar a lista e obter a posição final do pivô
        pivo_index = particao(lista, esq, dir)

        # se a posição do pivô for exatamente k, encontramos o k-ésimo menor elemento
        if pivo_index == k:
            return lista[pivo_index]

        # se a posição do pivô for maior que k, então o elemento procurado está na parte esquerda da lista
        elif pivo_index > k:
            return quickselect(lista, esq, pivo_index - 1, k)
        else:                                                # caso contrário, o elemento está na parte direita da lista
            return quickselect(lista, pivo_index + 1, dir, k)

    return None         # caso a busca não encontre o elemento (situação limite)
```

## 12.3. - Teste

[13]
✓ 0s

```python
# TESTANDO
import time
import random

# TESTANDO
listas = [
    ("lista ordenada", list(range(1, 21, 1))),
    ("lista ordenada", list(range(20, 41, 1))),
    ("lista invertida", list(range(60, 0, -2))),
    ("lista invertida", list(range(80, 0, -3))),
    ("lista aleatória", random.sample(range(1, 100), 20)),
    ("lista aleatória", random.sample(range(1, 100), 20))
]

for i, (nome, lista) in enumerate(listas, start=1):

    print(f"{i} - {nome}: {lista}")

    # ---------- QUICK SORT ----------
    lista_qs = lista.copy()
    comparacoes = 0
    copias = 0

    inicio = time.perf_counter()
    quick_sort(lista_qs)
    tempo_qs = time.perf_counter() - inicio

    print(" QuickSort")
    print(f"\tTempo: {tempo_qs:.8f} s")
    print(f"\tComparações: {comparacoes}")
    print(f"\tCópias: {copias}")


    # ---------- QUICK SELECT ----------
    lista_qselect = lista.copy()
    comparacoes = 0
    copias = 0

    k = len(lista_qselect) // 2  # mediana

    inicio = time.perf_counter()
    mediana = quickselect(lista_qselect, 0, len(lista_qselect) - 1, k)
    tempo_qselect = time.perf_counter() - inicio

    print(" QuickSelect")
    print(f"\tMediana: {mediana}")
    print(f"\tTempo: {tempo_qselect:.8f} s")
    print(f"\tComparações: {comparacoes}")
    print(f"\tCópias: {copias}")
    print()
```

```
1 - lista ordenada: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
 QuickSort
        Tempo: 0.00006407 s
        Comparações: 190
        Cópias: 399
 QuickSelect
        Mediana: 11
        Tempo: 0.00002994 s
        Comparações: 145
        Cópias: 155

2 - lista ordenada: [20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40]
 QuickSort
        Tempo: 0.00005191 s
        Comparações: 210
        Cópias: 440
 QuickSelect
        Mediana: 30
        Tempo: 0.00002832 s
        Comparações: 165
        Cópias: 176

3 - lista invertida: [60, 58, 56, 54, 52, 50, 48, 46, 44, 42, 40, 38, 36, 34, 32, 30, 28, 26, 24, 22, 20, 18
 QuickSort
        Tempo: 0.00009217 s
        Comparações: 435
        Cópias: 899
 QuickSelect
        Mediana: 32
        Tempo: 0.00006327 s
        Comparações: 435
        Cópias: 240

4 - lista invertida: [80, 77, 74, 71, 68, 65, 62, 59, 56, 53, 50, 47, 44, 41, 38, 35, 32, 29, 26, 23, 20, 17
 QuickSort
        Tempo: 0.00006454 s
        Comparações: 351
        Cópias: 728
 QuickSelect
        Mediana: 41
        Tempo: 0.00004725 s
        Comparações: 351
        Cópias: 196

5 - lista aleatória: [32, 63, 78, 85, 24, 26, 59, 21, 84, 34, 75, 72, 64, 27, 66, 95, 67, 53, 48, 28]
 QuickSort
        Tempo: 0.00002329 s
        Comparações: 81
        Cópias: 177
 QuickSelect
        Mediana: 63
        Tempo: 0.00000542 s
        Comparações: 46
        Cópias: 11

6 - lista aleatória: [77, 78, 71, 5, 3, 41, 10, 36, 22, 99, 67, 11, 32, 1, 58, 64, 6, 96, 25, 13]
 QuickSort
```